Project 7 (in Java): Array implementation of partial-order tree: the heap-sort.

What you need to do:

**You will be given 2 data files: data1 and data2.
1) Draw an illustration of array implementation of insertion and deletion of a partial-order tree, as taught in class.
*** Use a blank paper to draw the illustration as follows:
a) Write "Output:" on top of the paper, for the output during step f) below. Then, write all integers in data1 in one text-line data1 below it.
b) Draw an empty array size of number of data +1, with index written on the top of each cell of the array; write -1 in array [0], meaning no data in array.
c) Perform one insertion:
- Read one data from data1
- increate ary[0] by 1
- place data in ary[ary[0]], after the last data in array
- perform bubble-up process (draw arrow from spot to spot's father and do swap if needed.)
d) Draw a new array from the result of c.
e) Repeat c) and d) until all data are inserted.

f) Perform one deletion:
- Output the data at the root, ary[1]
- replace ary[1] with the last data in ary
- decrement ary[0] by 1
- perform bubble-down (draw arrows from spot to its left and right children, and do swap if needed.)
g) Draw a new array from the result of the last deletion in f).
h) repeat steps f) and g) until array is empty, i.e., array[0] == -1.

3) Implement your program according to the specs below. Debug your program until it passes compilation.
4) Run and debug your program with data1, until sorted data in outFile is the same as in your illustration.
5) Then, run your program with data2.
==========

Include in your hard copy (word file):
- cover page (include only main () algorithm steps, -1 otherwise.)
- illustration (-2 if omitted.)
- source code
- data1 // Caption: *** Below is data1
- outFile for data1 // Caption: *** Below is outFile for data1
- logFile for data1 // Caption: *** Below is logFile for data1
- data2 // Caption: *** Below is data2
- outFile for data2 // Caption: *** Below is outFile for data2
- logFile for data2 // Caption: *** Below is logFile for data2
****

Language: Java
****

Project Name: heapSort
Project points: 10 pts
Due Date: (10/10) on time: 11/11/2026 Wednesday before midnight (11:59pm)
(-10/10) non-submission: 11/11/2026 Wednesday after midnight

All submission MUST include soft copy (*.zip) and hard copy (word file) in the same email attachments with correct email subject as stated in the project submission requirement; otherwise, your submission will be rejected. -10/10.

Email subject: (313) first name last name <Project 7: Heap-sort (Java)>

*** Inside the email body includes:
- Your answer to the five questions given at the end of the project submission requirements. (-1 if omitted.)
- Screen recoding link. (-2 if omit screen recording).
*** Place your screen recording in your project submission email body below the 5 questions.

****
I. Inputs:

a) inFile (args [0]): a file contains integers, not in particular format.

II. Output:

a) outFile (args [1]): for sorted input data and others as specs dictates.

b) logFile (args [2]): as specs dictates.

****

III. Data Structures:

- heapSort class:
  - (int) count
  - (int []) heapAry // a 1-D integer array size of count +1, need dynamically allocated and initialize to 0.

  methods:
  - constructor (…)
  - countData(...) // Re-use code from previous projects.
  - buildHeap (...) //See algorithm below.
  - dumpHeap (...) //See algorithm below.
  - bubbleUp (...) // See algorithm below.
  - bubbleDown (...) // See algorithm below.
  - (int) findMinChild (heapAry, leftChild, rightChild) //See algorithm below.
  - mySwap (ary, i, j) // The method swaps ary[i] and ary[j]. On your own,
  - printHeap (heapAry, oFile) // output from heapAry [0] to heapAry [heapAry [0]] to oFile, where oFile
    //can be outFile or logFile. The printing format is as follows:
    // for example, if heapAry [0] is 4, print as follows.

Index	heapAry[index]	
0	4	// Indicating there are 4 data in heapAry
1	?	//? is whatever in heapAry [1]
2	?	//? is whatever in heapAry [2]
3	?	
4	?	

****

IV. main (…)

****

Step 0: check args length is 3 and check all files, one-by-one, it each can be opened.

inFile ← open input file from args [0] // Define inFile as Scanner.

outFile, logFile ← open from args [1] and args [2] // define both as PrinterWriter

//i..e.,

```
try (Scanner inFile = new Scanner(new BufferedReader(new FileReader(args[0])));
     PrintWriter outFile = new PrintWriter (new BufferedWriter (new FileWriter(args[1])));
     PrintWriter logFile = new PrintWriter (new BufferedWriter (new FileWriter(args[2])))) {
```

Step 1: count ← countData (inFile, logFile)

Step 2: heapAry ← dynamically allocate, size of count+1

Step 3: close inFile

Step 4: re-open inFile

Step 5: buildHeap (inFile, heapAry, logFile)

Step 6: outFile ← "In main () printing heapAry, after buildHeap" //use outFile.println("…")

printHeap (heapAry, outFile)

Step 6: dumpHeap (heapAry, outFile, logFile)

outFile ← "In main () printing heapAry, after dumpHeap"

printHeap (heapAry, outFile)

Step 7: exit

```
*****************************************
V. buildHeap (inFile, heapAry, logFile)
*****************************************
Step 1: logFile ← "Entering buildHeap () method." //use logFile.println("…")
Step 2: data ← get a data from inFile
Step 3: heapAry [0] ++
Step 4: if heapAry [0] <= count
                heapAry[heapAry[0]] ← data
        else
                cout << "Error! heap is full! This should not happen; your program has bugs."
                exit (-1)
Step 5: bubbleUp (heapAry)
Step 6:  logFile ← "in buildHeap (), printing heapAry, after bubbleUp (data)" //print data
         printHeap (heapAry, logFile)
Step 7: repeat Step 2 to Step 6 while inFile is NOT empty
Step 8: logFile ← "Leaving buildHeap () method."

*****************************************
VI. bubbleUp (heapAry, logFile)
*****************************************
Step 0: logFile ← "Entering bubbleUp () method."
Step 1: spot ← heapAry [0]
Step 2: father ← spot / 2
Step 3: if spot > 1 and father >= 1and heapAry[spot] < heapAry[father]
                mySwap (heapAry, spot, father)
                spot ← father
                father ← spot /2
                logFile ← "in bubbleUp method, printing heapAry one step up."
                printHeap (heapAry, logFile) // with caption: "in bubbleUp method, printing heapAry one step up."
Step 4: repeat Step 3 until condition failed
Step 5: logFile ← "Leaving bubbleUp () method."

*****************************************
VII. dumpHeap (heapAry, outFile, logFile)
*****************************************
Step 0: logFile ← "Entering dumpHeap () method."
Step 1: outFile ← heapAry[1]  // output the data in root.
Step 2: heapAry[1] ← heapAry[heapAry[0]]
Step 3: heapAry[0] - -
Step 4: bubbleDown (heapAry, logFile)
Step 5: logFile ← "in dumpHeap(), printing heapAry, after bubbleDown."
         printHeap (heapAry, logFile)
Step 6: repeat Step 1 to Step 5 while heapAry[0] > 0
Step 7: logFile ← "Leaving dumpHeap () method."
```

****
VIII. bubbleDown (heapAry, logFile):
****
Step 0: logFile ← "Entering bubbleDown () method."
Step 1: spot ← 1 // Begins from the root.
Step 2: leftChild ← spot * 2
        rightChild ← spot * 2 + 1
Step 3: minChild ← findMinChild (heapAry, leftChild, rightChild, logFile)
Step 4: if minChild <= heapAry[0] and spot >= 1 and heapAry[minChild] < heapAry[spot]
                mySwap (heapAry, minChild, spot)
                spot ← minChild
                printHeap (heapAry, logFile) // with caption: "in bubbleDown method, printing heapAry down one."
Step 5: repeat Step 2 to Step 4 until condition failed
Step 6: logFile ← "Leaving bubbleDown () method."

****
IX. (int) findMinChild (heapAry, leftChild, rightChild, logFile)
****
Step 0: logFile ← "Entering findMinChild () method."
Step 1: if leftChild > heapAry[0] // leftChild is out of range.
                minIndex ← leftChild
        else {
                if rightChild > heapAry[0] // leftChild is not out of range, but rightChild is out of range
                        minIndex ← leftChild
                else {
                        if heapAry[leftChild] <= heapAry[rightChild]
                                minIndex ← leftChild
                        else
                                minIndex ← rightChild
                    }
            }
Step 2: logFile ← "Leaving findMinChild () method, returning minIndex = " // write minInex
Step 3: return minIndex